Proiect TSS Bancomat - Vasile Cristian-Ioan

https://github.com/Cristiv06/TSS_Proiect_Bancomat

Specificația programului

Programul simulează un bancomat care procesează cereri de retragere de numerar. Toate cerințele (1–7) sunt rezolvate pe o singură metodă, `retrage` din clasa `ro.tss.bancomat.Bancomat`:

```
Rezultat retrage(int suma, int sold, int retrasAzi, boolean premium, int[] stoc)
```

Parametru	Semnificație
`suma`	suma cerută, în lei
`sold`	soldul contului, în lei
`retrasAzi`	suma retrasă deja în ziua curentă, în lei
`premium`	dacă titularul are cont premium
`stoc`	numărul de bancnote disponibile de 200, 100, 50 și 10 lei (vector de 4 întregi)

Precondiții (nu sunt verificate): `sold ≥ 0`, `retrasAzi ≥ 0`, `stoc` are 4 elemente nenegative.

Reguli, aplicate în ordine; prima regulă încălcată determină rezultatul:

1. Suma trebuie să fie între 10 și 5000 de lei și multiplu de 10, altfel `SUMA_INVALIDA`.
2. Un client standard poate retrage cel mult soldul; un client premium are un descoperit de 1000 de lei, deci cel mult `sold + 1000`. Altfel `FONDURI_INSUFICIENTE`.
3. `retrasAzi + suma` nu poate depăși 5000 de lei (standard) sau 10000 de lei (premium), altfel `LIMITA_DEPASITA`.
4. Suma se compune din cât mai multe bancnote mari (200, 100, 50, 10), în limita stocului. Dacă nu se poate compune, `BANCNOTE_INSUFICIENTE`.
5. Altfel `SUCCES`, împreună cu numărul de bancnote eliberate din fiecare valoare.

Ieșire: un obiect `Rezultat` cu starea retragerii și vectorul bancnotelor eliberate (doar zerouri dacă starea nu este `SUCCES`).

Apel	Rezultat
`retrage(380, 1000, 0, false, {5,5,5,5})`	`SUCCES` 1×200, 1×100, 1×50, 3×10
`retrage(1500, 500, 0, true, {10,10,10,10})`	`SUCCES` 7×200, 1×100 (folosește descoperitul)
`retrage(500, 2000, 4800, false, {10,10,10,10})`	`LIMITA_DEPASITA` (4800 + 500 > 5000)

Cerința 1. Date de test generate din specificație

1a. Partiționare de echivalență

Clasele intrărilor:

- **suma:** S_1 = {10 ≤ suma ≤ 5000, suma % 10 = 0} (valid); S_2 = {suma < 10}; S_3 = {suma > 5000}; S_4 = {10 ≤ suma ≤ 5000, suma % 10 ≠ 0}
- **sold:** F_1 = {suma ≤ sold}; F_2 = {sold < suma ≤ sold + 1000} (tratat diferit pentru premium); F_3 = {suma > sold + 1000}
- **premium:** P_1 = {true}; P_2 = {false}
- **retrasAzi:** L_1 = {retrasAzi + suma ≤ limita}; L_2 = {retrasAzi + suma > limita} (limita = 5000 standard, 10000 premium)
- **stoc:** T_1 = {permite compunerea sumei}; T_2 = {nu permite}

Clasele ieșirilor: i_1 = `SUMA_INVALIDA`, i_2 = `FONDURI_INSUFICIENTE`, i_3 = `LIMITA_DEPASITA`, i_4 = `BANCNOTE_INSUFICIENTE`, i_5 = `SUCCES`. Fiecare ieșire este produsă de cel puțin o clasă de echivalență.

Clasele de echivalență se obțin combinând clasele intrărilor.

Toate combinațiile ar fi 4 × 3 × 2 × 2 × 2 = **96**. Majoritatea dispar din cauza **ordinii regulilor**:

- S_2, S_3, S_4 dau mereu SUMA_INVALIDA, oricare ar fi celelalte intrări. Rămân 3 clase.
- F_3 dă mereu fonduri insuficiente, iar F_2 la fel pentru clientul standard. Nu mai contează L sau T.
- L_2 dă mereu limită depășită, deci T nu mai contează.

Rămân 15 clase de echivalență („plin” = stoc {100, 100, 100, 100}; „insuficient” = stoc {1, 0, 0, 0}):

Clasă	Combinație	Date de test (suma, sold, retrasAzi, premium, stoc)	Rezultat așteptat
C_2	S_2	(−100, 10000, 0, nu, plin)	`SUMA_INVALIDA`
C_3	S_3	(6000, 10000, 0, nu, plin)	`SUMA_INVALIDA`
C_4	S_4	(1234, 10000, 0, nu, plin)	`SUMA_INVALIDA`
C_{1111}	S_1 F_1 P_2 L_1 T_1	(500, 2000, 0, nu, plin)	`SUCCES` 2×200, 1×100
C_{1112}	S_1 F_1 P_2 L_1 T_2	(500, 2000, 0, nu, insuficient)	`BANCNOTE_INSUFICIENTE`
C_{112}	S_1 F_1 P_2 L_2	(500, 2000, 4800, nu, plin)	`LIMITA_DEPASITA`
C_{1211}	S_1 F_1 P_1 L_1 T_1	(500, 2000, 0, da, plin)	`SUCCES` 2×200, 1×100
C_{1212}	S_1 F_1 P_1 L_1 T_2	(500, 2000, 0, da, insuficient)	`BANCNOTE_INSUFICIENTE`
C_{122}	S_1 F_1 P_1 L_2	(500, 2000, 9800, da, plin)	`LIMITA_DEPASITA`
C_{2211}	S_1 F_2 P_1 L_1 T_1	(500, 100, 0, da, plin)	`SUCCES` 2×200, 1×100
C_{2212}	S_1 F_2 P_1 L_1 T_2	(500, 100, 0, da, insuficient)	`BANCNOTE_INSUFICIENTE`
C_{222}	S_1 F_2 P_1 L_2	(500, 100, 9800, da, plin)	`LIMITA_DEPASITA`
C_{21}	S_1 F_2 P_2	(500, 100, 0, nu, plin)	`FONDURI_INSUFICIENTE`
C_{32}	S_1 F_3 P_1	(2000, 100, 0, da, plin)	`FONDURI_INSUFICIENTE`
C_{31}	S_1 F_3 P_2	(2000, 100, 0, nu, plin)	`FONDURI_INSUFICIENTE`

1b. Analiza valorilor de frontieră

Pentru fiecare clasă de la 1a am testat valorile de pe frontieră și vecinii lor. 0 și 5010 sunt multipli de 10, deci testează doar ieșirea din interval. Unde nu se specifică altfel: sold = 10000, retrasAzi = 0, client standard, stoc plin.

Frontieră	Date de test	Rezultat așteptat
suma = 10	suma = 0 / 9 / 10 / 11	INV / INV / SUCCES 1×10 / INV
suma = 5000	suma = 4990 / 5000 / 5001 / 5010	SUCCES 24×200, 1×100, 1×50, 4×10 / SUCCES 25×200 / INV / INV
suma ≤ sold (standard)	suma = 500, sold = 500 / 499	SUCCES / FONDURI_INSUFICIENTE
suma ≤ sold + 1000 (premium)	suma = 1500, sold = 500 / 499; suma = 1000, sold = 0	SUCCES / FONDURI_INSUFICIENTE; SUCCES
limita 5000 (standard)	suma = 500, retrasAzi = 4500 / 4501	SUCCES / LIMITA_DEPASITA
limita 10000 (premium)	suma = 500, retrasAzi = 5000 / 9500 / 9501	SUCCES / SUCCES / LIMITA_DEPASITA
stoc (suma 380)	stoc {1,1,1,3} / {1,1,1,2} / {0,0,0,0} / {0,0,0,38}	SUCCES / BANCNOTE_INSUFICIENTE / BANCNOTE_INSUFICIENTE / SUCCES 38×10
înlocuirea bancnotelor	suma = 200, stoc {0, 2, 0, 0}	SUCCES 2×100

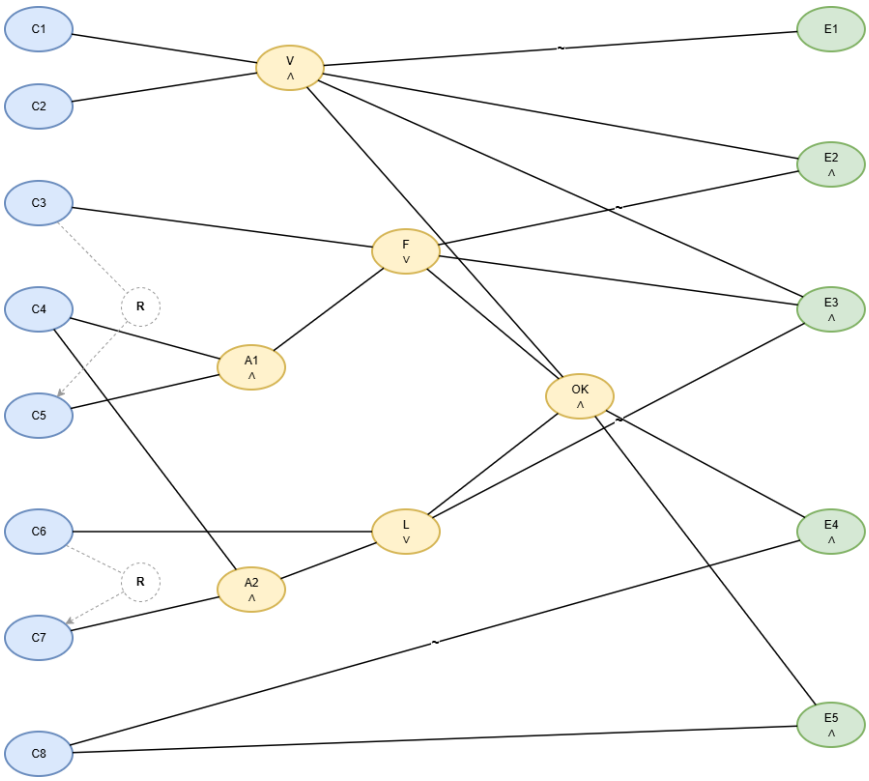
INV = SUMA_INVALIDA. Valorile separate prin „/” sunt teste distincte: în total 23 de teste.

1c. Graful cauză-efect

Cauze: C1: 10 ≤ suma ≤ 5000; C2: suma % 10 = 0; C3: suma ≤ sold; C4: premium; C5: suma ≤ sold + 1000; C6: retrasAzi + suma ≤ 5000; C7: retrasAzi + suma ≤ 10000; C8: stocul permite compunerea sumei.

Efecte: E1 = SUMA_INVALIDA, E2 = FONDURI_INSUFICIENTE, E3 = LIMITA_DEPASITA, E4 = BANCNOTE_INSUFICIENTE, E5 = SUCCES.

Relații (cu noduri intermediare): V = C1 ∧ C2 (sumă validă), F = C3 ∨ (C4 ∧ C5) (fonduri), L = C6 ∨ (C4 ∧ C7) (limită), OK = V ∧ F ∧ L; E1 = ¬V, E2 = V ∧ ¬F, E3 = V ∧ F ∧ ¬L, E4 = OK ∧ ¬C8, E5 = OK ∧ C8. **Constrângeri:** C3 R C5, C6 R C7.



Cauze
C1: 10 ≤ suma ≤ 5000
C2: suma % 10 == 0
C3: suma ≤ sold
C4: client premium
C5: suma ≤ sold + 1000
C6: retrasAzi + suma ≤ 5000
C7: retrasAzi + suma ≤ 10000
C8: stocul permite compunerea sumei

Noduri intermediare
V = C1 ∧ C2 (suma valida)
F = C3 ∨ (C4 ∧ C5) (fonduri suficiente)
L = C6 ∨ (C4 ∧ C7) (in limita zilnica)
OK = V ∧ F ∧ L

Efecte
E1 = ~V : SUMA_INVALIDA
E2 = V ∧ ~F : FONDURI_INSUFICIENTE
E3 = V ∧ F ∧ ~L : LIMITA_DEPASITA
E4 = OK ∧ ~C8 : BANCNOTE_INSUFICIENTE
E5 = OK ∧ C8 : SUCCES

Constrangeri: C3 R C5, C6 R C7
~ = NOT, ∧ = AND, ∨ = OR

Figura 1. Graful cauză-efect (∧ = AND, ∨ = OR, ~ = NOT, R = requires).

Tabelul de decizie („–” = nu contează) și datele de test corespunzătoare:

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11
C1	0	1	1	1	1	1	1	1	1	1	1
C2	–	0	1	1	1	1	1	1	1	1	1
C3	–	–	0	0	1	1	1	1	0	1	0
C4	–	–	0	1	0	1	0	0	1	1	1
C5	–	–	–	0	1	1	1	1	1	1	1
C6	–	–	–	–	0	0	1	1	1	0	1
C7	–	–	–	–	–	0	1	1	1	1	1
C8	–	–	–	–	–	–	0	1	1	1	0
Efect	E1	E1	E2	E2	E3	E3	E4	E5	E5	E5	E4

Test	Date de test	Test	Date de test
T1	(0, 10000, 0, nu, plin)	T7	(500, 2000, 0, nu, insuficient)
T2	(155, 10000, 0, nu, plin)	T8	(500, 2000, 0, nu, plin)
T3	(500, 100, 0, nu, plin)	T9	(500, 100, 0, da, plin)
T4	(2000, 100, 0, da, plin)	T10	(500, 2000, 6000, da, plin)
T5	(500, 2000, 4800, nu, plin)	T11	(500, 100, 0, da, insuficient)
T6	(500, 2000, 9800, da, plin)		

Implementarea în JUnit

Testele sunt scrise cu JUnit 5, câte o clasă pentru fiecare tehnică, ca să poată fi rulate separat: `EchivalentaTest` (15 teste), `FrontieraTest` (23), `CauzaEfectTest` (11). Fiecare test compară rezultatul complet (starea și bancnotele) cu cel calculat din specificație.

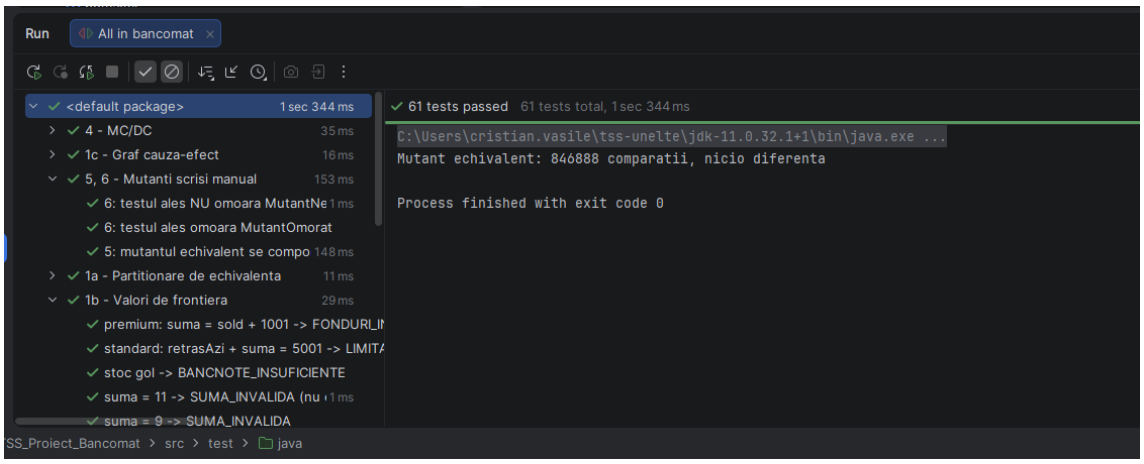


Figura 2. Toate testele proiectului rulate în IntelliJ IDEA: 61 de teste trecute.

Cerința 2. Acoperirea seturilor de teste

Am folosit **JaCoCo 0.8.16**, integrat în Maven. Fiecare set a fost rulat **separat** (de exemplu `mvn clean test -Dtest=FrontieraTest`); `clean` șterge datele rulării anterioare. Măsurătorile sunt pentru clasa `Bancomat`. Rapoartele HTML sunt în proiect, în rapoarte/acoperire/<set>/.

Set de teste	Teste	Instrucțiuni	Ramuri	Linii
1a Echivalență	15	114/114 (100%)	24/24 (100%)	25/25 (100%)
1b Frontieră	23	114/114 (100%)	24/24 (100%)	25/25 (100%)
1c Cauză-efect	11	114/114 (100%)	23/24 (95,8%)	25/25 (100%)

```
26.    public Rezultat retrage(int suma, int sold, int retrasAzi, boolean premium, int[] stoc) {
27.        if (suma < SUMA_MIN || suma > SUMA_MAX || suma % 10 != 0) {                  // [1]
28.            return Rezultat.eroare(Stare.SUMA_INVALIDA);                             // [2]
29.        }
30.        if (!(suma <= sold || (premium && suma <= sold + DESCOPERIT_PREMIUM))) {     // [3]
31.            return Rezultat.eroare(Stare.FONDURI_INSUFICIENTE);                      // [4]
32.        }
33.        int limita = LIMITA_STANDARD;                                                // [5]
34.        if (premium) {                                                               // [5]
35.            limita = LIMITA_PREMIUM;                                                 // [6]
36.        }
37.        if (retrasAzi + suma > limita) {                                             // [7]
38.            return Rezultat.eroare(Stare.LIMITA_DEPASITA);                           // [8]
39.        }
40.        int[] bancnote = new int[DENOMINATII.length];                                // [9]
41.        int rest = suma;                                                             // [9]
42.        int i = 0;                                                                   // [9]
43.        while (i < DENOMINATII.length && rest > 0) {                                 // [10]
44.            int nr = rest / DENOMINATII[i];                                          // [11]
45.            if (nr > stoc[i]) {                                                      // [11]
46.                nr = stoc[i];                                                        // [12]
47.            }
48.            bancnote[i] = nr;                                                        // [13]
49.            rest = rest - nr * DENOMINATII[i];                                       // [13]
50.            i++;                                                                     // [13]
51.        }
52.        if (rest != 0) {                                                             // [14]
53.            return Rezultat.eroare(Stare.BANCNOTE_INSUFICIENTE);                     // [15]
54.        }
55.        return Rezultat.succes(bancnote);                                            // [16]
56.    }                                                                                // [17]
```

Figura 3. Raportul JaCoCo pentru setul 1c: linia 27 (galben) este acoperită parțial.

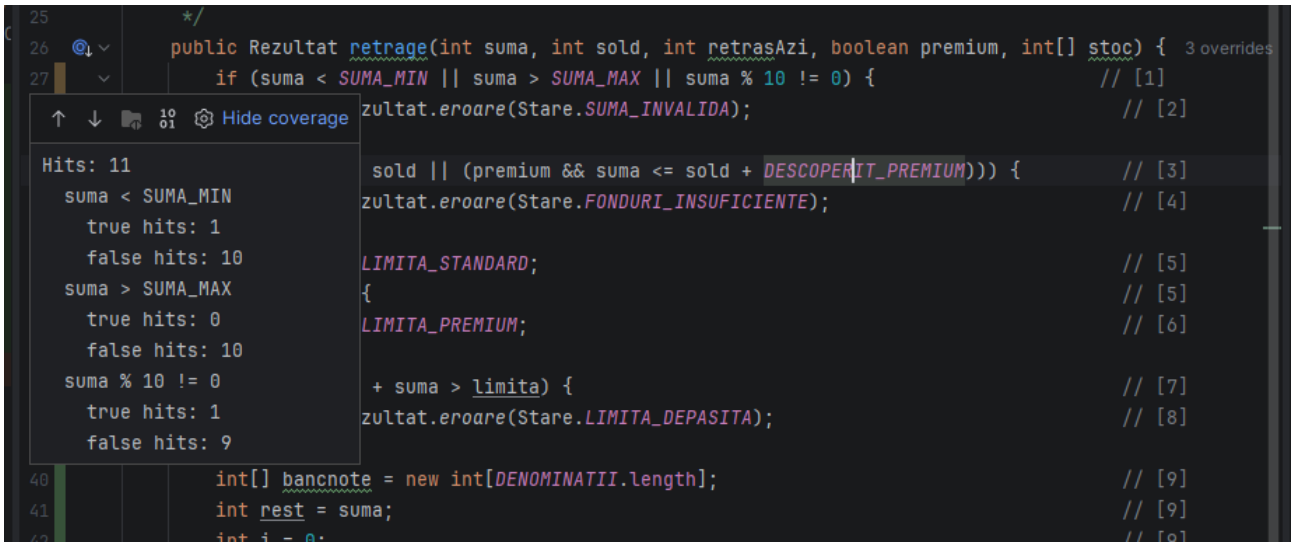


Figura 4. Același rezultat în IntelliJ IDEA: `suma > SUMA_MAX` nu este niciodată adevărată (true hits: 0).

Ramuri. JaCoCo măsoară ramurile în bytecode, unde fiecare condiție atomică a unei decizii compuse devine un salt separat: 12 condiții atomice dau 24 de ramuri. 100% înseamnă deci că fiecare condiție atomică a fost și adevărată, și falsă.

Cauză-efect ratează o ramură. Graful tratează intervalul 10 ≤ suma ≤ 5000 ca o singură cauză (C1), iar testul pentru C1 = 0 (suma = 0) verifică doar partea de jos. Partiționarea de echivalență separă S_2 de S_3, iar frontiera testează 5001 și 5010, deci ambele acoperă și `suma > 5000`.

Acoperirea nu diferențiază echivalența de frontieră. Ambele ating 100% pe toate măsurile, deși frontiera are 23 de teste și echivalența 15. Diferența apare abia la mutation testing (cerința 7): mutanții de tip < → <= sunt omorâți doar de valorile aflate exact pe frontieră.

Cerința 3. Graful orientat și complexitatea ciclomatică

```
public Rezultat retrage(int suma, int sold, int retrasAzi, boolean premium, int[] stoc) {
    if (suma < SUMA_MIN || suma > SUMA_MAX || suma % 10 != 0)              // [1]
        return Rezultat.eroare(Stare.SUMA_INVALIDA);                        // [2]
    if (!(suma <= sold || (premium && suma <= sold + DESCOPERIT_PREMIUM)))  // [3]
        return Rezultat.eroare(Stare.FONDURI_INSUFICIENTE);                 // [4]
    int limita = LIMITA_STANDARD;                                           // [5]
    if (premium)                                                            // [5]
        limita = LIMITA_PREMIUM;                                            // [6]
    if (retrasAzi + suma > limita)                                          // [7]
        return Rezultat.eroare(Stare.LIMITA_DEPASITA);                      // [8]
    int[] bancnote = new int[DENOMINATII.length];                           // [9]
    int rest = suma;                                                        // [9]
    int i = 0;                                                              // [9]
    while (i < DENOMINATII.length && rest > 0) {                            // [10]
        int nr = rest / DENOMINATII[i];                                     // [11]
        if (nr > stoc[i])                                                   // [11]
            nr = stoc[i];                                                   // [12]
        bancnote[i] = nr;                                                   // [13]
        rest = rest - nr * DENOMINATII[i];                                  // [13]
        i++;                                                                // [13]
    }
    if (rest != 0)                                                          // [14]
        return Rezultat.eroare(Stare.BANCNOTE_INSUFICIENTE);                // [15]
    return Rezultat.succes(bancnote);                                       // [16]
}                                                                           // [17]
```

Constante: `SUMA_MIN = 10`, `SUMA_MAX = 5000`, `DESCOPERIT_PREMIUM = 1000`, `LIMITA_STANDARD = 5000`, `LIMITA_PREMIUM = 10000`, `DENOMINATII = {200, 100, 50, 10}`.

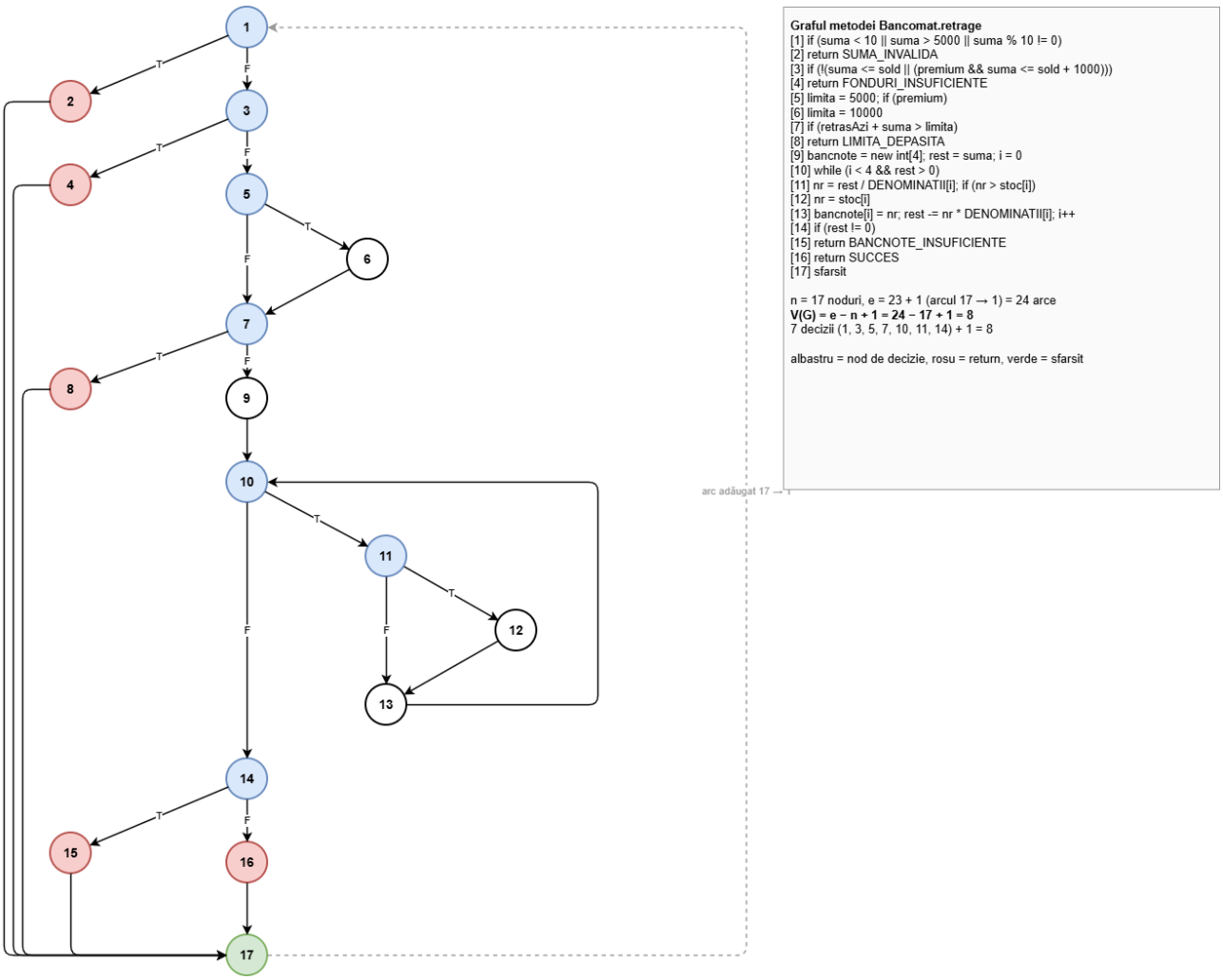


Figura 5. Graful metodei (fișierul draw.io: `docs/graf_bancomat.drawio`). Albastru = decizie, roșu = `return`; arcul punctat 17 → 1 face graful tare conex.

Graful are n = 17 noduri și 23 de arce; cu arcul adăugat 17 → 1, e = 24:

$$V(G) = e - n + 1 = 24 - 17 + 1 = 8$$

Verificare: 7 noduri de decizie (1, 3, 5, 7, 10, 11, 14) + 1 = 8. **Set de 8 circuite liniar independente** care acoperă toate arcele (fiecare introduce cel puțin un arc nou):

	Circuit	Arce noi
a	1, 2, 17, 1	1→2, 2→17, 17→1
b	1, 3, 4, 17, 1	1→3, 3→4, 4→17
c	1, 3, 5, 7, 8, 17, 1	3→5, 5→7, 7→8, 8→17
d	1, 3, 5, 6, 7, 8, 17, 1	5→6, 6→7
e	1, 3, 5, 7, 9, 10, 14, 16, 17, 1	7→9, 9→10, 10→14, 14→16, 16→17
f	10, 11, 13, 10	10→11, 11→13, 13→10
g	10, 11, 12, 13, 10	11→12, 12→13
h	1, 3, 5, 7, 9, 10, 14, 15, 17, 1	14→15, 15→17

Circuitele e și h nu pot fi executate singure (o sumă validă intră cel puțin o dată în buclă); la execuție apar combinate cu circuitele buclei f și g. De exemplu, suma = 100 cu stoc plin parcurge e + 2·f.

Cerința 4. Set de teste MC/DC

MC/DC cere ca fiecare condiție și fiecare decizie să ia ambele valori, iar fiecare condiție să influențeze independent decizia (schimbând doar acea condiție, decizia se schimbă). Deciziile compuse ale metodei:

- **D1** (nod 1) = A ∨ B ∨ C; A: suma < 10, B: suma > 5000, C: suma % 10 ≠ 0
- **D2** (nod 3) = A ∨ (B ∧ C); A: suma ≤ sold, B: premium, C: suma ≤ sold + 1000
- **D5** (nod 10) = A ∧ B; A: i < 4, B: rest > 0

Test	Date de test (suma, sold, retrasAzi, premium, stoc)	Rezultat
M1	(100, 1000, 0, nu, plin)	`SUCCES 1×100`
M2	(0, 1000, 0, nu, plin)	`SUMA_INVALIDA`
M3	(6000, 1000, 0, nu, plin)	`SUMA_INVALIDA`
M4	(15, 1000, 0, nu, plin)	`SUMA_INVALIDA`
M5	(500, 100, 0, nu, plin)	`FONDURI_INSUFICIENTE`
M6	(500, 100, 0, da, plin)	`SUCCES 2×200, 1×100`
M7	(1500, 100, 0, da, plin)	`FONDURI_INSUFICIENTE`
M8	(100, 1000, 4950, nu, plin)	`LIMITA_DEPASITA`
M9	(100, 1000, 0, nu, stoc gol)	`BANCNOTE_INSUFICIENTE`

D1	A	B	C	D1		D2	A	B	C	D2
M1	F	F	F	F		M1	T	F	T	T
M2	T	F	F	T		M5	F	F	T	F
M3	F	T	F	T		M6	F	T	T	T
M4	F	F	T	T		M7	F	T	F	F

D1: perechile M1/M2, M1/M3 și M1/M4 arată influența lui A, B, respectiv C. Valorile alese fac și condițiile neevaluate (scurtcircuitate) să aibă aceeași valoare ca în M1 (0 % 10 = 0, 6000 % 10 = 0). **D2:** M1/M5 → A, M6/M5 → B, M6/M7 → C (4 teste pentru 3 condiții, minimul n + 1).

D5: în M1, prima iterație dă (T, T), iar după ce restul devine 0 dă (T, F) → influența lui B; în M9, cu stoc gol, bucla se termină cu i = 4 și rest = 100, adică (F, T) → influența lui A. **Deciziile simple** (D3 = premium, D4 = limită depășită, D6 = nr > stoc[i], D7 = rest ≠ 0) sunt adevărate în M6, M8, M9, respectiv M9 și false în M1. Setul de 9 teste (`MCDCTest`) satisface deci MC/DC.

Cerința 5. Mutant de ordinul 1 echivalent

Mutantul (operator ROR, nodul 14) modifică o singură instrucțiune:

```
Original:  if (rest != 0)
Mutant:    if (rest > 0)
```

Este echivalent pentru că `rest` nu poate deveni negativ: pornește de la `suma ≥ 10` și la fiecare iterație scade cu `nr · DENOMINATII[i]`, unde `nr ≤ rest / DENOMINATII[i]`, deci scăderea nu depășește `rest`. Pentru `rest ≥ 0`, condițiile `rest != 0` și `rest > 0` sunt identice, deci mutantul se comportă la fel ca originalul pe orice intrare.

Verificare: testul `MutantiTest` compară originalul cu mutantul pe 846.888 de combinații de intrări, fără nicio diferență. Major (cerința 7) a generat automat același mutant (id 94) și niciun set de teste nu l-a omorât.

Cerința 6. Mutant omorât și mutant neomorât

Testul ales (cerința 1b): `retrage(10, 10000, 0, false, plin)` → `SUCCES [0, 0, 0, 1]`.

Mutant (nodul 1)	Rezultat pe testul ales	Omorât?
`suma < SUMA_MIN` → `suma <= SUMA_MIN`	`SUMA_INVALIDA`	Da
`suma > SUMA_MAX` → `suma >= SUMA_MAX`	`SUCCES` [0, 0, 0, 1]	Nu

Primul mutant respinge suma 10, deci rezultatul diferă și testul îl omoară. Al doilea dă același rezultat pentru suma 10, dar **nu este echivalent**: pentru suma = 5000 originalul întoarce `SUCCES`, iar mutantul `SUMA_INVALIDA`. Testul ales verifică doar frontiera de jos, deci nu îl omoară. Ambele afirmații sunt verificate automat în `MutantiTest`.

Cerința 7. Mutation testing cu PIT și Major

Am folosit **PIT 1.30.1** și **Major 3.0.1** și am calculat scorul pentru testele de la cerința 1 (cele trei seturi împreună) și pentru testele MC/DC de la cerința 4, cu formula din curs:

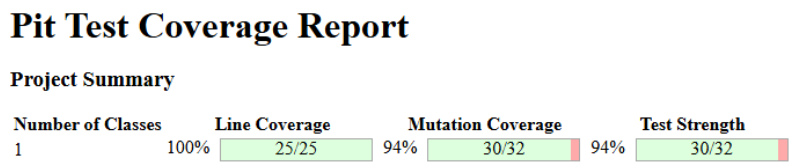
$$MS(T) = D / (L + D)$$

D = mutanți omorâți, L = mutanți vii **neechivalenți**. Mutanții echivalenți au fost identificați prin analiza codului și excluși. Rapoartele complete sunt în proiect, în rapoarte/pit/ și rapoarte/major/.

7.1 PIT

PIT generează mutanții în **bytecode** și rulează testele pe fiecare mutant, direct din Maven (`mvn test-compile org.pitest:pitest-maven:mutationCoverage -DtargetTests=...`). Cu operatorii impliciți (DEFAULTS: ConditionalsBoundary, RemoveConditional, Math, Increments, NullReturnVals) a generat **32 de mutanți**, dintre care 2 echivalenți:

- `rest > 0` → `rest >= 0` în `while`: când rest = 0, iterațiile în plus au nr = 0 și nu schimbă nimic;
- `nr > stoc[i]` → `nr >= stoc[i]`: când nr = stoc[i], atribuirea `nr = stoc[i]` nu schimbă valoarea.



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	100%	25/25	94%	30/32	94%	30/32

Figura 6. Raportul PIT pentru testele de la cerința 1: 30 din 32 de mutanți omorâți.

7.2 Major

Major este un plugin pentru compilatorul `javac` 11 care generează mutanții pe **codul sursă** (arborele sintactic), cu operatorii ROR, AOR, COR, STD, LVR și EVR. Toți mutanții sunt compilați într-o singură clasă, iar mutantul activ se alege la rulare prin `Config.__M_NO`. Analizorul inclus în Major suportă doar JUnit 3/4, așa că am scris un analizor echivalent (`MajorAnaliza.java`) care, pentru fiecare mutant, setează `__M_NO` și rulează testele JUnit 5; mutantul e omorât dacă un test eșuează (inclusiv prin timeout, pentru buclele infinite).

Major a generat **97 de mutanți**, dintre care 9 echivalenți: cei doi de la PIT, variante ale lui `rest > 0` (`rest > 1`, `rest > -1`, `rest != 0`, eliminarea condiției), `i < 4` → `i != 4`, `suma % 10 != 0` → `suma % 10 > 0` și mutantul de la cerința 5.

Rezultate

Set de teste	PIT	Major
Cerința 1 (1a + 1b + 1c)	30/30 = 100%	88/88 = 100%
Cerința 4 (MC/DC)	24/30 = 80,0%	76/88 = 86,4%

Pe seturi separate: echivalență 80,0% (PIT) / 88,6% (Major), frontieră 100% / 97,7%, cauză-efect 83,3% / 88,6%.

Mutanții neechivalenți rămași vii:

- **Mutanții de frontieră** (< → <= la sumă, sold și limită) supraviețuiesc în MC/DC, echivalență și cauză-efect, care folosesc valori din interiorul claselor; îi omoară doar setul de frontieră.
- **Limita premium** (`if (premium)` forțat fals): niciun test MC/DC sau de echivalență nu are un client premium între 5000 și 10000 retrași. MC/DC verifică fiecare decizie separat, nu și interacțiunea dintre decizii.
- **COR din Major** (`premium && X` → `premium == X`, `suma <= sold + 1000` → ==): scapă setului de frontieră, ale cărui teste premium sunt exact pe frontiera descoperitului; îi omoară valorile interioare din partiționarea de echivalență. Frontiera și echivalența se completează, cum spune și cursul.

Diferențele dintre scoruri

1. **Nivelul mutației.** PIT lucrează pe bytecode, unde o decizie compusă a devenit salturi separate, deci mută doar comparații individuale. Major lucrează pe sursă și operatorul COR înlocuiește expresii logice întregi; acești mutanți cer combinații de condiții, de aceea frontiera are 100% la PIT, dar 97,7% la Major.
2. **Constantele.** Constantele `static final` (10, 5000, 1000) sunt copiate în bytecode, iar PIT le poate muta (InlineConstant, în grupul ALL). Major vede doar numele lor (`SUMA_MAX`), care nu sunt literali, deci nu le mută.
3. **Operatorii și echivalenții.** Major generează mulți mutanți ușor de omorât (AOR: fiecare operator aritmetic înlocuit cu toți ceilalți; STD: ștergerea instrucțiunilor), ceea ce crește scorurile, dar și mai mulți mutanți echivalenți (9 față de 2). Scorurile absolute nu se compară direct între unelte; contează ordinea seturilor.

Concluzie. Ambele unelte dau aceeași ordine: testele de la cerința 1 omoară toți mutanții neechivalenți, iar MC/DC are scorul cel mai mic, deși atinge acoperire maximă cu cele mai puține teste (9).

Utilizarea instrumentelor AI

Instrument folosit: **Claude** (Claude Code, aplicația desktop).

Scop și solicitare	**Rezultat**	**Modificări și verificare**
Implementarea programului și a testelor. Ideea temei (retragere de la bancomat) am ales-o eu, apoi am cerut implementarea: „Da, mergem pe bancomat cu specificația propusă. Fă folderul pe Desktop și implementează.”	Codul, testele JUnit, mutanții manuali	Am rulat testele în IntelliJ IDEA: toate cele 61 trec (Figura 2). Am verificat de mână rezultatele unor apeluri și am notat pe foaie clasele de echivalență și valorile de frontieră.
Verificarea acoperirii și a grafurilor, redactarea documentației. Exemple de solicitări: „Deschid în IntelliJ și ne uităm acolo, îmi zici ce am de verificat”; „Vreau să confirmi dacă sunt așa la graf: când e o structură de decizie e și alternativa, orice structură repetitivă are un ciclu, nodul final reunește ieșirile finale ale programului”; „E proiect de o persoană și trebuie să ating doar esențialul, de unde crezi că mai putem scurta?”	Rapoartele JaCoCo, grafurile, textul	Acoperirea verificată și în IntelliJ (Figura 4), cu aceleași rezultate ca JaCoCo. Am refăcut pe foaie graful programului și graful cauză-efect și am verificat complexitatea ciclomatică. Am scurtat și am editat documentația.
Generarea și analiza mutanților cu a doua unealtă: „Descarcă Major 3.0.1 de pe mutation-testing.org și JDK 11 Temurin și fă partea 7.2 cu Major.”	Configurația PIT și Major, analizorul, rapoartele	Am parcurs argumentele pentru mutanții echivalenți și pentru cei de la cerința 6. Rapoartele au fost regenerate după ultima modificare a codului.